

The Crumb Doctrine

M. Drake Stapleton

Version 1.4, 2026-10-02

1 Preamble

We are building machines that will be trusted with real work, and trust cannot rest on a model's confident prose or on a human's careful review, because both fail. A model will invent a formula that was never in the sources. A human will stamp PASS on a test that never ran. The Crumb Doctrine exists to fix the meaning of what the machine knows in something harder than words: verification receipts. A crumb is the smallest unit of knowledge we allow ourselves to stand on, and this document says what a crumb is, what it demands, and what it forbids.

The doctrine is not yet a property the machine enforces. What we can say today is narrower: admission and closure are now fail-closed. Nothing enters the Crumblin without a receipt, and any step the machinery cannot verify reports itself as not run, never as passed.

2 Definitions

Crumb. A verified program plus its verification receipt. Code plus qualifying evidence. A program without a receipt is a claim, not a crumb. A receipt without a program is paperwork, not a crumb. The two travel together or not at all.

Receipt. The evidence that a program does what it claims. It records what was checked, how it was checked, when it was checked, which verifier did the checking, and what the program was checked against. A receipt must be recheckable by an independent party under fully declared prerequisites, without asking us for permission or for our word. If an independent party cannot recheck it, it is not a receipt. A receipt makes its evidence inspectable and recheckable. Its specification, verifier, assumptions, and applicability remain open to challenge.

A receipt records at minimum: the digest of the verifier that ran, the digest of the specification it checked against, the closure of dependencies the result depends on, the execution environment, the architecture assumptions, the randomness and seeds, the resource bounds, the timeout semantics, the nondeterminism policy, the input corpus or a commitment to it, and the exact result. A receipt that omits any of these is incomplete. When a crumb is scored in Turings, the measurement profile fields travel with or alongside the receipt, so the two do not drift.

Provenance. A crumb's history, carried with it permanently: where the crumb came from, what generated it, what it was verified against, and every hand it passed through. Provenance is never stripped, never summarized away, never "cleaned up." A crumb with no history is a rumor.

Crumbline. The append-only lineage of verified crumbs and their outcomes. Empirical claims requiring generalization are verified against appropriately sealed held-out evidence. The guarantee is simple. Models built the laboratory, the generators, the harness, the machinery. Any lineage claimed as independent machine discovery must carry auditable evidence that prohibited generators and learners had no access to the sealed side of the curriculum. That is what makes the records trustworthy: they are firsthand.

3 The Tenets

1. **Claims are admitted through verification, not prose.**

Semantics defines behavior. Specifications identify claims. Receipts record evidence for those claims under explicit assumptions. Policy determines admissibility; authorization remains separate. A type-checking receipt, a fuzzing receipt, a bounded model-checking result, and a formal proof do not establish the same thing.

Rationale: if two receipts look alike, the claims they record are still different; admissibility has to be decided on the claim, not on the shape of the receipt.

2. **No receipt, no crumb.**

An unverified program may be useful, interesting, or promising, but it is not a crumb and it does not enter the library. There is no provisional shelf, no “verified enough,” no trust-me tier.

Rationale: one exception becomes the standard; the line holds only if it never bends.

3. **Every receipt must be recheckable by an independent party.**

If verifying a crumb requires our machines, our keys, or our word, the receipt is theater. An independent party with the crumb, its receipt, and the prerequisites the receipt declares must be able to rerun the check and reproduce the declared result within its declared equivalence or tolerance conditions. Deterministic receipts may declare exact equality as their tolerance.

Rationale: trust that depends on us is rent; trust that survives without us is property.

4. **In experiments claiming independent rediscovery, the learner is never told why a crumb matters.**

AIEN sees observations only: inputs and outputs, no titles, no labels, no explanations, no hint that one crumb matters more than another. The task boundary comes from the runtime, never from descriptive text. This rule governs the rediscovery claim; it is not a restriction on all learning. What counts as evidence of rediscovery is stated in the Blind Curriculum Protocol.

Rationale: if the machine claims to have rediscovered something true on its own, we must be able to tell rediscovery from quiet coaching, and that is only possible if we never coached it in that experiment.

5. **Provenance is permanent.**

Every crumb carries its full history forever: generator, seed, verifier, date, and what it was checked against. Nothing is stripped for convenience.

Rationale: a result you cannot trace is a result you cannot defend; when something breaks, the history is how you find out why.

6. **The Crumbline is append-only and unbroken.**

Crumbs are added; they are never edited in place, never silently replaced, never deleted to hide

a failure. Failures are recorded as failures, alongside the passes.

Rationale: a lineage with gaps is a story, not evidence; the misses teach as much as the hits, but only if they stay on the record.

7. **Weights suggest; programs explain.**

A model may propose anything. Proposals become knowledge only when they survive verification and enter the Crumblin as crumbs. Until then they are suggestions with no standing.

Rationale: this is the division of labor the whole project rests on; blurring it is how slop gets in.

8. **Verification is independent of generation.**

The thing that checks a crumb must not be the thing that made it. Complete assumption independence is impossible: generator and verifier will share instruction-set semantics, arithmetic definitions, parsers, perhaps even the semantic representation itself. We require enumerated independence instead: separate implementation where it matters, separate failure modes, and every shared assumption stated explicitly in the receipt as a limitation of what the receipt can claim.

Rationale: Omega is the system; verification is performed by an independent verifier, never by the generator. Independence is what makes a receipt worth anything, and honesty about where independence ends is what makes independence real.

4 The Composition Principle

Large programs are built only from verified crumbs. A crumb may be composed with other crumbs, and the composition itself becomes a crumb only when the composed program is verified and receipted in turn. Nothing unverified enters the chain at any level: not the foundation, not the middle, not the top.

This is the long-term shape of the work. The Omega language's module and import system is the crumb store: to build on something in Omega is to build on a verified crumb, the way Rust code is built on crates. The difference is that a crate is trusted by reputation while a crumb is trusted by receipt. In that language, the doctrine stops being a document and becomes the grammar. The easy way to build and the right way to build are the same act.

Until that language exists, the principle is enforced the hard way: by gates. See the closing.

5 The Learner Principle

The autonomous improvement process may acquire durable machine-generated knowledge only through the Crumblin. Where the claim is independent rediscovery, the learner is shown crumbs as observations only and is never shown the sealed side: no generator identities, no difficulty scores, no provenance, no labels, no explanations of what any crumb is for. This blindness is mandatory specifically for independent-rediscovery claims; it is not a restriction on all learning. The segregation is structural, not cosmetic. It is what makes every later claim of independent discovery checkable: either the machine found it on its own from observations, or we can point to the exact place we told it.

Observations alone are not enough to guarantee blindness. Information can leak through ordering,

repetition frequency, curriculum selection, input shape, timing, dependency structure, naming, rarity, or which crumbs are exposed at all. We therefore require a Blind Curriculum Protocol: a specification that enumerates exactly which channels are observable to the learner and tests each channel for leakage. Until that protocol exists and is enforced, no claim of independent rediscovery counts as evidence.

Where independent rediscovery is claimed, the learner’s ignorance of why is not a limitation to be fixed later. It is the mechanism that keeps the whole lineage honest.

6 What the Doctrine Demands of Tooling

A doctrine that lives in a document is a wish. A doctrine that lives in the merge gate is a law. The Crumb Doctrine therefore demands the following of every tool in the pipeline, and a tool that cannot meet them does not get to touch crumbs:

1. **Gates, not guidelines.** Storing the crumb must be a required check before a change is accepted, not a recommendation in a style guide. If the check is optional, it will be skipped under pressure, and pressure is constant.
2. **Automatic minting.** When a program passes verification, its crumb is minted by the machinery itself, through the existing promotion and provenance modules. No manual step, no extra effort, no excuse. Anything that costs the author effort will be avoided.
3. **Independent verification.** The verifier runs separately from the generator, with its own assumptions stated. Shared assumptions are documented as a limitation of the receipt, not hidden.
4. **Visible counts.** The number of crumbs minted, verified, and failed is reported where people can see it without digging. What is measured is done; what is invisible is neglected.
5. **Fail closed.** Any step of the crumb pipeline that cannot be verified reports itself as not run, never as passed. A missing receipt is a missing crumb, and the system says so plainly.

The standard applies to our own tooling first. The day we found our own scripts stamping PASS without running the checks, the doctrine did not bend for us. It does not bend for anyone.

7 The Lifecycle

We give the toolchain four stages, and each stage establishes something different.

`omega check` establishes well-formedness and declared effects: the program is coherent and states the effects it claims. A workspace `omega build` produces an unsealed realization for testing: a thing that runs, carrying no admission into the Crumblin. `omega verify` produces claim-specific evidence against the specification the claim names: the receipt records what was checked and how. `omega seal` admits the artifact and its qualified dependency closure: from that point the artifact is addressable by its identifiers and its closure is on the record. Sealing does not settle the

cost of verification once and for all. Evidence can be reused while its assumptions, dependencies, environment, and policy remain applicable; rechecking and composition can incur additional costs.

Two rules hold across all four stages. A receipt attests to a claim, not to a program in the abstract: the same semantic object paired with a different specification is a different claim, and needs its own evidence. And execution receipts bind the actual tested realization and its environment, not merely the source semantic identity. What was run, where it was run, and under which conditions it ran belong to the receipt; the source identity alone does not say what executed.

Status: v1.4. This document states principles. The mechanisms are separate works (the Crumblin curriculum spec, the crumbs crate, the Omega language design): receipt-and-closure checking is implemented and exercised in CI; repository-wide closure enforcement and required-before-merge gating are not yet active; full language semantics are not. They stand or fall on their own evidence.

Revision note (v1.4): verification is no longer described as paid once at seal time. Evidence may be reused while its assumptions, dependencies, environment, and policy remain applicable; rechecking and composition can incur additional costs. Tenet 1 rationale no longer repeats the body. Enforcement state unchanged.